Day 10 — React.js Introduction

Answers & Practical Implementation

Objective: Create a basic React application using Vite, understand the project folders, and build reusable Header, Home, and Footer components using props.

1. What is React.js?

React.js is a JavaScript library used to build user interfaces, especially interactive web pages. It lets developers split a page into small, reusable pieces called components.

2. Advantages of React

• Reusable components reduce repeated code.

• A declarative approach makes UI code easier to understand.

• The Virtual DOM helps React update the interface efficiently.

• One-way data flow through props makes data relationships clearer.

• React has a large ecosystem and community.

3. Components, JSX, and Functional Components

Component: An independent, reusable part of the user interface, such as a header or footer.

JSX: A syntax extension that lets us write HTML-like markup inside JavaScript. In JSX, use `className` instead of HTML's `class`.

Functional component: A JavaScript function that returns JSX. Component names should start with a capital letter.

4. What are Props?

Props (short for properties) let a parent component pass data to a child component. Props are read-only inside the child.

```
function Header({ title }) {
  return <h1>{title}</h1>;
}

// Use the component:
<Header title="My React App" />
```

5. What is Vite?

Vite is a frontend build tool and development server. It provides a quick way to create and run a React project during development.

6. Create a React application using Vite

Prerequisite: Install a current supported version of Node.js. Open VS Code, then open Terminal → New Terminal and run these commands one at a time:

```
npm create vite@latest day10-react-app -- --template react
cd day10-react-app
npm install
npm run dev
```

Open the local URL shown in the terminal (commonly http://localhost:5173). Keep the terminal running while using the app.

7. Understanding the project structure

```
day10-react-app/
  public/          # Static files served as-is
```

```
src/            # Main application source code
  components/   # Reusable UI components (create this folder)
  App.jsx       # Main app component
  main.jsx      # Entry point that mounts React
  index.css     # Global styles
index.html      # HTML page used by Vite
package.json    # Project scripts and dependencies
```

The exact files can vary slightly by Vite version or template. Put React components and most application logic in src; use public for static assets such as a favicon.

8. Create Header, Home, and Footer components

Inside `src`, create a folder named `components`. Create the following three files.

src/components/Header.jsx

```
function Header({ title, subtitle }) {
  return (
    <header className="header">
      <h1>{title}</h1>
      <p>{subtitle}</p>
    </header>
  );
}

export default Header;
```

src/components/Home.jsx

```
function Home({ studentName, topic }) {
  return (
    <main className="home">
      <h2>Welcome, {studentName}!</h2>
      <p>Today's topic: {topic}</p>
      <p>This is my first React application created with Vite.</p>
    </main>
  );
}

export default Home;
```

src/components/Footer.jsx

```
function Footer({ year }) {
  return (
    <footer className="footer">
      <p>© {year} My React App</p>
    </footer>
  );
}

export default Footer;
```

9. Use the components and pass props

Replace the contents of `src/App.jsx` with:

```
import Header from './components/Header';
import Home from './components/Home';
import Footer from './components/Footer';
import './index.css';

function App() {
  return (
    <div className="app">
      <Header
        title="My React App"
        subtitle="Learning React step by step"
      />
      <Home studentName="Shweta" topic="React.js Introduction" />
      <Footer year={new Date().getFullYear()} />
    </div>
  );
}

export default App;
```

10. Add simple styling

Replace the contents of `src/index.css` with:

```
* {
  box-sizing: border-box;
}

body {
```

```
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f4f7fb;
  color: #243247;
}

.header, .home, .footer {
  text-align: center;
  padding: 22px;
}

.header {
  background: #dcecff;
}

.home {
  min-height: 55vh;
}

.footer {
  background: #e8edf3;
}
```

11. Expected output

When the app runs, the page displays a header titled “My React App” with the subtitle “Learning React step by step”, a welcome message for Shweta, the topic “React.js Introduction”, and a footer with the current year.

12. How data flows through props

In `App.jsx`, the parent passes values such as `title`, `studentName`, `topic`, and `year`. The child components receive these values in their function parameters and display them. Changing a passed value in App.jsx changes what the child displays.

13. Run and test the application

• Save all files in VS Code.

• Make sure the terminal is in the day10-react-app folder.

• Run npm run dev if the development server is not already running.

• Open the local URL printed in the terminal.

• Check that the header, welcome text, topic, and footer appear. Fix any spelling or import-path errors shown in the terminal.

Conclusion: A React application was created using Vite. Header, Home, and Footer were separated into reusable functional components, and props were used to pass data from App to each component.